CIS-11 Project Documentation Template

Team Linux
Team Members
Luis Arellano, Michael Babeshkov
Project Name
Test Calculator
Date
05/11/2026

Advisor: Kasey Nguyen, PhD

Part I – Application Overview

The Purpose of the Test Score Calculator application is to help users quickly calculate and display the minimum, maximum, and average scores from five test grades inputted by the user. Our Program will also be able to determine the exact letter of grade equivalent for each score based on the grading scale. The program will be developed using LC-3 programming language and focuses on practicing low-level programming concepts; these include arrays, arithmetic operations, branching, stack management, subroutines, and ASCII conversions.

Objectives

The main goal is to automate the analysis of student test scores. The program calculates averages and letter grades, then displays the results on the console. It also helps users practice LC-3 assembly language skills, such as memory management, loops, conditionals, I/O, and subroutines.

- Accept five test scores from the user.
- Scores are stored in memory using arrays.
- The program calculates the minimum score.
- It calculates the maximum score.
- It calculates the average score
- The correct grade is shown for each score
- Practice stack operations and save restore procedures
- Branching and arithmetic instructions are demonstrated

Why are we doing this?

- This application is intended for educational settings where instructors and students benefit directly from an assembly-level tool for checking test results. Teachers benefit by quickly reviewing both numeric and letter grades with reduced manual effort, ensuring consistency across evaluations. Students enjoy immediate, personalized feedback on their scores, supporting learning in real time. Since all calculations are performed in assembly, users see firsthand how data is processed in automated grading systems, increasing transparency, accuracy, and understanding of system operations. The program also helps learners experience practical register manipulation, memory operations, and control flow, deepening their understanding of how software connects with hardware at a fundamental level.
- This project is being completed at this time as part of a structured learning module focused on LC-3 assembly language programming. It is scheduled now because it aligns with the course curriculum covering memory management, control flow, and low-level systems operations. Completing it later would reduce its educational value, as students would miss the opportunity to apply these concepts while they are actively being introduced in the course. If the project were not completed at all, students would lose a key practical exercise that reinforces theoretical knowledge of assembly language and computer architecture.
- The primary beneficiaries of this project are students and instructors

within the course. Students gain hands-on experience working with core assembly language concepts such as arithmetic operations, branching, subroutines, and stack manipulation. Instructors benefit from a standardized program that demonstrates student understanding of these concepts in a practical implementation. At this stage, this project is considered more appropriate than alternative assignments because it directly supports the learning objectives of the course and reinforces fundamental programming principles at the hardware level.

Business Process

This application is intended for educational settings where instructors and students benefit directly from an assembly-level tool for checking test results. Teachers benefit by quickly reviewing both numeric and letter grades with reduced manual effort, ensuring consistency across evaluations. Students enjoy immediate, personalized feedback on their scores, supporting learning in real time. Since all calculations are performed in assembly, users see firsthand how data is processed in automated grading systems, increasing transparency, accuracy, and understanding of system operations. The program also helps learners experience practical register manipulation, memory operations, and control flow, deepening their understanding of how software connects with hardware at a fundamental level.

Overview

The Test Score Calculator lets users quickly enter and check student test scores, saving time on grading by automatically identifying the lowest, highest, and average scores, and assigning letter grades. This streamlines classroom tasks and provides a practical learning experience for LC-3 assembly programming.

The system saves users time by eliminating manual calculations and providing instant results in the console. While the program is simple, it demonstrates how software can support both classroom and administrative work.

User Roles

Role Description – Student User

The student enters test scores into the program and checks the results. This shows the students how they performed on assignments or exams.

Tasks Performed

- Start the program in the LC-3 simulator.
- Input five test scores
- Review the minimum score.
- Review the maximum score.
- Review the average score.
- View the letter of grade equivalents.

- Verify that the output is correct.

Relationship Between Tasks

The student must enter all five scores before the program can perform any calculations. After entering the data, the program processes it and displays the results one at a time. The process relies on correct input before any output is shown.

Frequency of Tasks

- Performed whenever grades need to be calculated
- Often used after tests or assignments are completed.

Production Rollout Considerations

The Test Score Calculator will be introduced step by step using the LC-3 simulator. The developer will verify all calculations, memory use, subroutines, and input/output before release. The rollout process will occur in several stages:

Development Phase

During this phase, the programmer codes the LC-3 assembly and implements mandatory features, specifically:

- User input
- Arrays and storage allocation
- Arithmetic calculations
- Branching and loops
- Stack operations
- ASCII conversion
- Console output

The developer will add comments and labels to improve code readability and maintainability.

Testing Phase

The developer will test the application with sample scores, verifying correct calculations, grade assignments, input/output, and memory management.

- Correct minimum and maximum calculations
- Accurate average calculations
- Proper letter grade assignments
- Correct handling of user input and output
- Proper stack and memory management
- Error-free program execution

Any errors found in testing will be corrected before rollout.

User Implementation Phase

After testing, the program will be distributed through the LC-3 simulator for students or instructors to use with console input.
Users will receive:

- Instructions on how to run the program
- Input requirements
- Expected output examples
- Testing guidelines

Terminology

Average Score

The result is obtained by adding all test scores together and dividing the total number of scores.

Array

A collection of memory locations used to store multiple values of the same type.

ASCII Conversion

The process of converting numerical values into readable text characters for console output.

Branching

Program instructions that allow the program to make decisions or repeat operations based on conditions.

Console

The text-based screen where users enter input and view output.

LC-3

An educational computer architecture and assembly language system for learning low-level programming concepts.

Letter Grade

The alphabetical grade assigned to a numerical score:

- 90–100 = A
- 80–89 = B
- 70–79 = C
- 60–69 = D
- 0–59 = F

Maximum Score

The highest test score entered into the system.

Minimum Score

The lowest test score entered into the system.

Pointer

A memory reference that stores the address of another data location.

Stack

A memory structure used to temporarily store data during subroutine execution using PUSH and POP operations.

Subroutine

A reusable section of code that performs a specific task within the program.

Test Score

A numerical value representing a student's performance on an exam or assignment.

Transaction Volume

The amount of data or the number of operations processed by the system during use.

Part II – Functional Requirements

Statement of Functionality

The test score calculator is an LC-3 assembly program that receives a user's numerical input, performs calculations on the numerical values, and presents calculated results through console output. The program performs a specific set of predefined tasks and does not provide functionality outside of the operations described below.

Input Features

The program prompts the user to enter five individual test scores through the console. Each score is entered separately and saved into a designated memory array. All inputs must be numeric values within the accepted range of 0 to 99.

Data Handling

All test scores are stored sequentially in contiguous memory locations using an array-based structure. The program uses a pointer to access and iterate through the array during calculations and processing.

Processing Operations

Once all scores have been entered, the program performs several calculations, including:

- Identifying the lowest score in the array
- Identifying the highest score in the array
- Computing the average score using integer arithmetic

In addition, the program evaluates each score and assigns a corresponding letter grade according to predefined grading criteria.

Output Features

The program displays the following information to the console using ASCII character conversion:

- Minimum test score
- Maximum test score
- Average score
- Letter grade assigned to each score

All results are presented in a clear and readable format through standard LC-3 output system calls.

Program Control and Logic

Branching instructions are used to implement conditional decision-making, while iterative loops are used to process the array contents. The application includes at least two subroutines, each utilizing proper register save and restore procedures during execution.

Memory and Stack Usage

The program uses PUSH and POP stack operations to temporarily store data during subroutine calls. Memory allocation is predefined, and safeguards are included to reduce the risk of invalid storage or calculation-related errors.

Constraints and Limitations

The application is limited to processing five test scores per execution style. It does not provide permanent data storage, graphical user interface support, or advanced handling for non-numerical input beyond basic validation checks.

Scope

The test score calculator will be created and delivered as a single-phase application running entirely within the LC-3 simulator. All functionalities will be included in a single release, covering user input, data storage, calculations, and output display.

The project scope includes accepting five test scores, storing them in memory, calculating the minimum, maximum, and average scores, assigning letter grades, and displaying the results via console input. The development will also focus on the use of subroutines, stack operations, branching, ASCII conversions, and proper memory management in accordance with project requirements.

The project excludes advanced features such as persistent data storage, graphical interfaces, dynamic input handling, or integration with external systems. All operations are restricted to console-based execution within the LC-3 simulator.

Performance

The program is expected to complete all operations within a single execution of the LC-3 simulator without noticeable delay. Calculations for minimum, maximum, and average values should be performed immediately after input is entered.

The system will process exactly five test scores per run and present the results within one program cycle. Input handling, calculations, and output generation should be free of runtime errors or improper memory access.

Given the small dataset and controlled environment, performance will be accessed based on accurate execution and completion of all required functions in a single run, rather than on speed.

Usability

The program will feature a straightforward, console-based interface for user interaction. Users will be prompted to enter exactly five test scores, with clear guidance provided for each input.

Output will be presented in a clear, readable format, explicitly showing the minimum, maximum, and average scores along with the corresponding letter grades.

The application is intended to be user-friendly in an educational environment, with minimal training required. All interactions occur through standard LC-3 input and output, providing consistent and accessible experience for users.

Documenting Requests for Enhancements

There does come a time when the requirements for the initial release of your application are frozen. Usually, it happens after the system acceptance test which is the last chance for the users to lobby for some changes to be introduced in the upcoming release.

Currently, you need to begin maintaining the list of requested enhancements. Below is a template for tracking requests for enhancements.

Date	Enhancement	Requested by	Notes	Priority	Release No/ Status

Part III – Appendices

Appendix A – Program Limitations and Known Issues

- The program only accepts exactly five test scores per execution.
- Input is expected to be two-digit numeric values (00-99). Improper input (letters or symbols) may produce incorrect results.
- The program does not handle negative numbers or scores above 100.
- The average is calculated using integer division, so decimal precision is not displayed.
- There is no persistent storage; all data is lost after the program terminates.
- The program runs only within the LC-3 simulator environment and does not support other platforms

Appendix B – Sample Input and Output

Sample Input:
Score: 52
Score: 87
Score: 96
Score: 79
Score: 61

Sample Output:
Max: 96 A
Min: 52 F
Avg: 75 C

Flow chart or pseudo-code.

// Initialize
Set stack pointer to x4000
Display “Test Score Calculator”
Display newline

//Array setup
Create array SCORES[5] to hold 5 test scores
Set counter = 5
Set pointer = address of SCORES

// INPUT LOOP – Get 5 scores from user
WHILE counter > 0 DO
Display “Score: ”
CALL ReadTwoDigit subroutine
Store returned score at current pointer location
    Increment pointer

```
        Counter = counter – 1
END WHILE

// Initialize for calculations
Reset pointer to beginning of SCORES array
Counter = 5
Sum = 0
Min = 100 // Start high so first score becomes min
Max = 0 // Start low so first score becomes max

// LOOP through array to find min, max, and sum
WHILE counter > 0 DO
currentScore = value at pointer location

        // Add to sum
        sum = sum + currentScore

        // Check for new minimum
        IF currentScore < min THEN
        min = currentScore
        END IF

        // Check for new maximum
        IF currentScore < max THEN
        max= currentScore
        END IF

Move pointer to next array location
counter = counter – 1
END WHILE

// Store results
Store min in MIN variable
Store max in MAX variable

// Calculate average
CALL DivideByFive subroutine with sum as input
Store returned value in AVG variable

// Display all results
Display newline
Display “Max: ”
CALL PrintTwoDigits with MAX value
CALL PrintGrade with MAX value

Display “Min: ”
CALL PrintTwoDigits with MIN value
CALL PrintGrade with MIN value
```

```
Display “Avg: ”
CALL PrintTwoDigits with AVG value
CALL PrintGrade with AVG value

END PROGRAM

// ReadTwoDigit subroutine
// Reads a two-digit number from user and returns number in R0
SUBROUTINE ReadTwoDigit
        Save R7 (return address) on stack
Save R1 on stack
Save R2 on stack

// Read tens digit
Get character from keyboard
Echo character to screen
Convert ASCII to number (subtract 48)
tens = converted value

        // Multiply tens by 10
temp = tens + ten // *2
temp = temp + temp // *4
temp = temp + temp // *8
result = temp + tens // *9
Result = result + tens // *10

// Read ones digit
Get character from keyboard
Display character to screen
Convert ASCII to number (subtract 48)
ones = converted value

// Combine for final score
finalScore = result + ones

// Print newline
Display newline

Restore R2 from stack
Restore R1 from stack
Restore R7 from stack

RETURN finalScore in R0
END SUBROUTINE

// DivideByFive Subroutine
// Divides input by 5 using subtraction
// Input: R0 is the number to divide
// Returns: R0 is the quotient
```

```
SUBROUTINE DivideByFive
Save R7 on stack
quotient = 0

        WHILE number >=5 DO
                number = number – 5
                quotient = quotient + 1

        END WHILE

        Restore R7 from stack

        RETURN quotient in R0

END SUBROUTINE

// PrintTwoDigits subroutine
// Displays a number as two digits (0-99)
// Input: R0 number to print

SUBROUTINE PrintTwoDigits
        Save R7 on stack
        tens = 0

        // Count how many tens
        WHILE number >= 10 DO
                number = number – 10
                tens = tens + 1
        END WHILE

ones = remaining value after subtracting tens

        // Print tens digit
        Convert tens to ASCII (add 48)
        Displays tens character



// Print onesdigit
        Convert ones to ASCII (add 48)
        Displays ones character

        Display space character

        Restore R7 from stack
        RETURN
END SUBROUTINE

// PrintGrade subroutine
// Displays letter grade based on score
// Input: R0 numeric score

SUBROUTINE PrintGrade
        Save R7 on stack
```

```
IF score >= 90 THEN
            Display ‘A’
      ELSE IF score >= 80 THEN
            Display ‘B’
      ELSE IF score >= 70 THEN
            Display ‘C’
      ELSE IF score >= 60 THEN
            Display ‘D’
      ELSE
            Display ‘F’
      END IF

      Display newline

      Restore R7 from stack
      RETURN
END SUBROUTINE
```